

LAPORAN RESMI PRAKTIKUM

DevSecOps — Bab 2 Konsep Container dan Instalasi Docker



Nama : Fawwaz Mufid Wardaya
Kelas : D4 Teknik Informatika B
NRP : 3126640050

**PROGRAM STUDI TEKNIK INFORMATIKA
DEPARTEMEN TEKNIK INFORMATIKA DAN KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2026**

Pendahuluan

Latar Belakang

Containerization mengemas aplikasi beserta dependensinya ke dalam unit yang berjalan konsisten. Pada Linux, container bukan VM dengan kernel sendiri, melainkan proses biasa yang memperoleh isolasi via namespace, pembatasan via cgroup, dan pengurangan privilege via mekanisme kernel. Container berbagi kernel host sehingga lebih ringan dan cepat, namun boundary isolasinya lebih lemah dari VM. Docker Engine mengelola lifecycle image/container via daemon yang mendelegasikan ke containerd dan runc; image bersifat read-only dengan beberapa layer, ditambah writable layer saat container berjalan. Praktikum Bab 2 menjalankan container pertama, inspeksi image, log, dan membangun image custom sederhana.

Rumusan Masalah

Mengapa tag latest tidak dianjurkan untuk deployment reproducible?

Apa peran containerd dan runc dalam arsitektur Docker?

Apa konsekuensi keamanan user masuk group docker?

Bandingkan layer nginx:1.26-alpine dan image custom yang dibangun.

Kapan sebaiknya memilih VM daripada container?

Pembahasan

Dasar Teori

Image tersusun atas layer read-only; setiap layer menyimpan perubahan, bukan salinan penuh. Copy-on-write (CoW) memungkinkan banyak container berbagi layer read-only dan hanya menyimpan perubahan lokal — mempercepat distribusi & mengurangi duplikasi storage, tetapi writable layer bersifat sementara. Image bersifat content-addressable: digest kriptografis mengidentifikasi content, tag adalah nama yang dapat dipindahkan. Alur docker run: tarik image → buat container → alokasikan fs read-write → buat network → mulai container. Container berhenti saat proses PID 1 selesai.

VM vs Container:

Aspek	Virtual Machine	Container
Kernel	Guest OS sendiri	Berbagi kernel host
Ukuran	Umumnya GB	MB–ratusan MB
Startup	Detik–menit	Detik/kurang
Isolasi	Boundary hypervisor lebih kuat	Lebih ringan, perlu hardening

Praktikum 1 — Verifikasi Docker & Container Pertama

Perintah:

```
docker version
docker run hello-world
```

```
└─ docker run hello-world
```

Hello from Docker!

This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:

1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub. (arm64v8)
3. The Docker daemon created a new container from that image which runs the executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it to your terminal.

To try something more ambitious, you can run an Ubuntu container with:

```
$ docker run -it ubuntu bash
```

Share images, automate workflows, and more with a free Docker ID:

<https://hub.docker.com/>

For more examples and ideas, visit:

<https://docs.docker.com/get-started/>

Gambar: docker run hello-world menghasilkan 'Hello from Docker!'

Analisis: hello-world membuktikan alur client→daemon→registry→container→output berfungsi. docker version menampilkan Client (darwin/arm64) dan Server Docker Desktop 4.89.0 (Engine 29.7.2, linux/arm64, containerd v2.3.3, runc 1.4.3).

Praktikum 2 — Container Nginx & Ubuntu

Perintah:

```
docker run -d --name web-public -p 8080:80 nginx:1.26
```

```
docker ps && docker logs --tail 20 web-public
```

```
curl http://localhost:8080
```

```
docker run --rm ubuntu:22.04 bash -c "cat /etc/os-release"
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
be659922b089	nginx:1.26	"/docker-entrypoint..."	4 seconds ago	Up 3 seconds	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp	web-public

Gambar: docker ps menampilkan container web-public berjalan (port 0.0.0.0:8080->80)

```

└─ curl http://localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>

```

Gambar: curl http://localhost:8080 mengembalikan halaman 'Welcome to nginx!'

Analisis: opsi -p 8080:80 menerbitkan port 80 container ke 8080 host — tanpa alamat bind yang dibatasi, port terekspos pada seluruh interface (0.0.0.0), risiko yang di produksi dimitigasi via firewall/reverse proxy. Container ubuntu diverifikasi via /etc/os-release (Ubuntu 22.04.5 LTS).

Praktikum 3 — Dockerfile Custom Web Statis

Perintah:

```

FROM nginx:1.26-alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
docker build -t pens-web:1.0 . && docker run -d --name pens-app -p 9090:80 pens-web:1.0
curl http://localhost:9090

```

```

└─ docker build -t pens-web:1.0 . && docker run -d --name pens-app -p 9090:80 pens-web:1.0
[+] Building 0.0s (1/1) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 2B
ERROR: failed to build: failed to solve: failed to read dockerfile: open Dockerfile: no such file or directory

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/ufr00xzhysb11fh1ob8x9ajmc

What's next:
  Debug this build failure with Gordon → docker ai "help me fix this build failure"

└─ curl http://localhost:9090
<h1>Docker Lab PENS</h1>
<p>Container berhasil berjalan.</p>

```

Gambar: docker build pens-web:1.0 dan curl http://localhost:9090 mengembalikan 'Docker Lab PENS'

Analisis: image pens-web:1.0 (9 layer, 77.1 MB) berbagi 8 layer pertama identik dengan nginx:1.26-alpine (8 layer, 77.9 MB) — hanya menambah 1 layer dari COPY index.html. Ini bukti nyata copy-on-write: tidak ada duplikasi filesystem penuh. nginx:1.26-alpine (77.9 MB) jauh lebih kecil dari nginx:1.26 Debian (283 MB), konfirmasi base image minimal mengurangi attack surface. EXPOSE 80 hanya metadata; penerbitan butuh -p 9090:80. Catatan keamanan: image berjalan sebagai root (User kosong) — harus dimitigasi di bab hardening.

Troubleshooting

Gejala	Penyebab	Tindakan
Port 8080/9090 dipakai	Container/layanan lain	docker ps; ganti port host atau hentikan container lama
Container berhenti segera	PID 1 exit non-nol	docker logs; nginx perlu 'daemon off;' di foreground
docker ps butuh sudo	User belum di group docker	usermod -aG docker \$USER; newgrp docker (atau rootless)

Evaluasi dan Latihan Mandiri

1. Mengapa tag latest tidak dianjurkan untuk deployment reproducible?

Tag latest adalah nama yang dapat dipindahkan; dua pull berbeda waktu bisa menghasilkan content berbeda. Reproducible butuh identitas content: tag spesifik (nginx:1.26) atau pinning digest (image@sha256:...). Pin tetap harus diperbarui saat base image dapat patch — reproducible bukan tak berubah, melainkan perubahan terversi & terverifikasi. Di praktikum ini dipakai nginx:1.26 (digest sha256:41b19...).

2. Peran containerd dan runc?

Daemon mendelegasikan lifecycle ke containerd (high-level runtime: image transfer, storage, eksekusi, network) dan runc (low-level runtime: membuat proses sesuai OCI Runtime Spec, menyiapkan namespace/cgroup/capabilities). Pemisahan ini penting untuk troubleshooting — kegagalan bisa dari API/daemon/containerd/runc/aplikasi. Tercatat containerd v2.3.3, runc 1.4.3.

3. Konsekuensi keamanan user di group docker?

Group docker setara akses root: user bisa docker run --privileged atau bind mount filesystem host (/ , /etc, socket Docker). Kompromi akun biasa → kompromi root host; group docker bukan least privilege. Mitigasi: batasi akses socket, jangan beri socket ke untrusted workload, gunakan rootless mode + MAC (AppArmor/SELinux).

4. Bandingkan layer nginx:1.26-alpine vs image custom?

nginx:1.26-alpine: 8 layer, 77.9 MB. pens-web:1.0: 9 layer, 77.1 MB — 8 layer pertamanya identik (digest sama) dengan base, hanya menambah 1 layer tipis dari COPY index.html. Tidak ada duplikasi fs; ukuran hampir sama. Bukti CoW: image custom hanya menumpuk perubahan. Bandingkan nginx:1.26 Debian (7 layer, 283 MB) — jauh lebih besar, attack surface lebih luas.

5. Kapan pilih VM daripada container?

Saat butuh isolasi melebihi container: (1) guest kernel berbeda/multi-OS — container berbagi kernel host; (2) regulasi/kompliansi butuh boundary hypervisor; (3) workload legacy untuk mesin penuh; (4) multi-tenancy tidak terpercaya (VM tegas terhadap exploit kernel). Container cocok untuk dev/test, microservices, CI/CD. Praktik modern: container di dalam VM agar dapat unit ringan + boundary infrastruktur tegas.

Penutup

Kesimpulan

Container adalah isolasi proses berbasis kernel yang berbagi kernel host — lebih ringan & cepat dari VM, namun boundary lebih lemah. Praktikum Bab 2 berhasil: hello-world (client→daemon→registry→container), nginx:1.26 pada port 8080 diverifikasi via curl, ubuntu:22.04 via os-release, dan image custom pens-web:1.0 dibangun & diverifikasi pada port 9090. Inspeksi layer membuktikan CoW: pens-web (9 layer, 77.1 MB) berbagi 8 layer identik

dengan nginx:1.26-alpine (77.9 MB). Catatan keamanan: port 0.0.0.0 terekspos dan image berjalan sebagai root — dimitigasi di bab hardening berikutnya.

Daftar Pustaka

Materi Ajar DevOps (DevSecOps Terintegrasi) — Bab 2. Politeknik Elektronika Negeri Surabaya, 2026.

Docker Inc. (2026). Docker Documentation — Engine, CLI, Images. <https://docs.docker.com/>

Open Container Initiative. Image, Runtime, Distribution Spec. <https://github.com/opencontainers/>

Docker Inc. (2026). Docker Engine Security. <https://docs.docker.com/engine/security/>

Akses terakhir: 8 September 2026.